

Exqutor 논문 상세 정리

논문 제목: Exqutor: Extended Query Optimizer for Vector-augmented Analytical Queries

저자: Hyunjoon Kim, Chaerim Lim, Hyeonjun An (연세대학교 BDAI Lab), Rathijit Sen (Microsoft Gray Systems Lab), Kwanghyun Park (연세대학교)

출처: arXiv:2512.09695v2 [cs.DB], 2025년 12월 11일

1. 논문이 해결하고자 하는 문제 (Introduction)

1.1 배경: RAG 파이프라인과 벡터 유사도 검색

최근 대규모 언어 모델(LLM)을 전자상거래, 의료, 추천 시스템 등에 통합하기 위해 **RAG(Retrieval-Augmented Generation)** 워크플로우가 널리 사용되고 있다. RAG는 LLM이 답변을 생성할 때, 외부 지식(문서, 이미지 등)을 먼저 검색해서 참고하게 함으로써 "환각(hallucination)" 문제를 줄이는 기술이다.

이 RAG 파이프라인에서 핵심적인 역할을 하는 것이 **벡터 유사도 검색(Vector Similarity Search, VSS)**이다. 쉽게 말해, 사용자가 질문하면 그 질문을 수치 벡터로 변환하고, 데이터베이스에 미리 저장된 문서/이미지 벡터들 중 가장 비슷한 것을 찾아서 LLM에 전달하는 것이다.

1.2 문제: 벡터 검색이 포함된 복잡한 분석 쿼리

기존 RAG는 단순히 "가장 비슷한 문서 top-k개 찾기" 수준이었다. 하지만 실제 현업에서는 훨씬 복잡한 요구가 있다. 예를 들어:

"2024년 데이터를 사용해서, 주어진 헤드폰 이미지와 시각적으로 유사한 부품을 공급하는 업체를 찾고, 관련 고객 댓글과 할인율 정보를 함께 분석해 최적의 할인율을 제안해줘."

이런 질문을 SQL로 변환하면, 벡터 유사도 검색뿐만 아니라 **여러 테이블 간의 조인(JOIN)**, **날짜 필터링**, **정렬** 등 관계형 연산이 함께 필요하다. 이처럼 벡터 검색과 관계형 연산을 결합한 쿼리를 논문에서는 **VAQ(Vector-Augmented Analytical Query)**라고 부른다.

1.3 핵심 문제: 부정확한 카디널리티 추정

데이터베이스는 쿼리를 실행하기 전에 **쿼리 옵티마이저**가 "어떤 순서로, 어떤 방식으로 데이터를 처리할지" 실행 계획을 세운다. 이때 각 연산의 결과가 몇 건이나 나올지를 추정하는 것을 **카디널리티 추정(Cardinality Estimation)**이라고 한다.

문제는 기존 데이터베이스 시스템들이 벡터 유사도 검색의 카디널리티를 제대로 추정하지 못한다는 것이다. 논문의 Table 1에 따르면:

- **pgvector**: 항상 전체 행 수의 33.3%로 고정 추정
- **VBASE**: 항상 전체 행 수의 50.0%로 고정 추정
- **DuckDB**: 항상 전체 행 수의 100.0%로 추정 (즉, 필터 효과가 없다고 가정)

이렇게 실제와 동떨어진 추정을 하면, 옵티마이저가 잘못된 실행 계획을 세우게 되어 쿼리 실행 시간이 크게 늘어난다. 예를 들어, 벡터 검색으로 200건만 나오는데 시스템이 "8000만 건 전부 나올 것"이라고 추정하면, 불필요하게 큰 해시 조인(Hash Join)을 선택하는 등 비효율적인 계획을 세우게 된다.

1.4 Exqutor의 제안

이 논문은 **Exqutor**라는 프레임워크를 제안한다. Exqutor는 기존 데이터베이스 시스템의 쿼리 옵티마이저에 "플러그인"처럼 끼워 넣을 수 있는 확장 모듈로, 벡터 유사도 검색의 카디널리티를 정확하게 추정해서 더 좋은 실행 계획을 세울 수 있게 해준다.

2. 배경 지식 (Background)

2.1 벡터 데이터베이스의 두 가지 유형

전문(특화) 벡터 데이터베이스: Milvus, Pinecone, Qdrant, Manu, ChromaDB 등. 벡터 검색에 최적화되어 있지만, 복잡한 SQL 쿼리(조인, 집계 등)를 지원하기 어렵다. 단일 컬렉션(테이블) 위에서 메타데이터 필터링 정도만 가능하다.

범용 벡터 데이터베이스: pgvector(PostgreSQL 기반), VBASE(PostgreSQL 기반), AnalyticDB, DuckDB-VSS(DuckDB 기반) 등. 기존 관계형 데이터베이스에 벡터 검색 기능을 추가한 형태이다. 여러 테이블을 조인하고, 복잡한 분석 쿼리를 수행할 수 있어 VAQ 실행에 더 적합하다.

Exqutor는 이 **범용 벡터 데이터베이스**를 대상으로 한다.

2.2 최근접 이웃 검색과 벡터 인덱스

벡터 유사도 검색의 핵심은 **최근접 이웃 검색(Nearest Neighbor Search)**이다:

- **KNN(K-Nearest Neighbor):** 모든 벡터와 하나씩 비교해서 정확한 결과를 반환하지만, 데이터가 많으면 매우 느리다.
- **ANN(Approximate Nearest Neighbor):** 미리 구축한 인덱스를 활용해 근사적으로 빠르게 검색한다. 정확도를 약간 희생하지만 속도가 훨씬 빠르다.

대표적인 ANN 인덱스 기법으로 **HNSW(Hierarchical Navigable Small World)**가 있다. HNSW는 계층적 그래프 구조를 만들어서, 검색할 때 그래프를 탐색하며 빠르게 유사한 벡터를 찾는다. 이 논문에서도 HNSW를 주요 벡터 인덱스로 사용한다.

2.3 ECQO(Exact Cardinality Query Optimization)

ECQO는 쿼리 실행 계획을 세울 때, 추정이 아닌 **실제로 쿼리 일부를 실행해서 정확한 결과 건수를 얻는** 기법이다. 정확한 카디널리티를 알면 최적의 실행 계획을 세울 수 있지만, 계획 수립 단계에서 실제 쿼리를 실행하므로 오버헤드가 발생한다. 기존에는 이 오버헤드 때문에 "오프라인 기법"으로만 여겨졌지만, Exqutor는 벡터 인덱스 검색이 매우 빠르다는 점을 활용해 이를 온라인에서 효율적으로 사용한다.

3. 동기 부여 (Motivation) — 왜 이 연구가 필요한가

3.1 범용 벡터 DB의 VAQ 성능 문제

논문은 pgvector에서 실제 VAQ를 실행한 예시를 보여준다(Figure 2). partsupp 테이블에 8000만 건의 DEEP 데이터셋 벡터 임베딩이 저장된 상황에서:

- **기존 실행 계획:** 옵티마이저가 벡터 검색의 선택도를 잘못 추정해서, 불필요한 Full Table Scan과 Hash Join을 선택. 실행 시간이 매우 길다.
- **최적 실행 계획:** 벡터 검색의 실제 카디널리티를 알면, Index Scan과 Nested Loop Join을 선택할 수 있어 실행 시간이 수 자릿수(orders of magnitude) 줄어든다.

3.2 근본 원인

pgvector, VBASE, DuckDB 모두 벡터 유사도 검색 연산자(VSS operator)에 대해 **고정된 선택도(selectivity) 값**을 사용한다. 쿼리의 조건이나 데이터 분포와 무관하게 항상 같은 비율로 추정하기 때문에, 실제 결과 건수와 큰 차이가 발생한다.

4. Vector-augmented SQL Analytics 벤치마크 (Section IV)

4.1 벤치마크 설계 목적

기존 ANN 벤치마크(예: ANN Benchmarks)는 단일 테이블에서 top-k 검색 성능만 평가한다. 하지만 VAQ는 여러 테이블 간 조인, 필터링, 집계를 포함하므로, 이를 제대로 평가할 벤치마크가 없었다.

이 논문은 최초의 VAQ 전용 벤치마크를 제안한다. 기존에 널리 사용되는 TPC-H와 TPC-DS 벤치마크를 확장하여 벡터 데이터를 추가한 것이다.

4.2 TPC-H 확장

TPC-H의 **partsupp** 테이블에 두 개의 임베딩 컬럼을 추가한다:

- **ps_image_embedding** : 이미지 임베딩 벡터
- **ps_text_embedding** : 텍스트 임베딩 벡터
- **ps_tag** : 태그 정보 (멀티모달 특성을 시뮬레이션)

또한 **part** 테이블에도 **p_text_embedding** 컬럼을 추가한다.

사용된 벡터 데이터셋:

- **DEEP** (96차원): 딥러닝 기반 이미지 특징 벡터
- **SIFT** (128차원): 이미지의 지역 특징 벡터
- **SimSearchNet++** (256차원): 유사 이미지 탐지용 벡터
- **YFCC** (192차원): 멀티미디어 연구용 벡터
- **WIKI** (768차원): 위키피디아 텍스트 임베딩

TPC-H 쿼리 중 Q3, Q5, Q8, Q9, Q10, Q11, Q12, Q20을 선택하여 벡터 유사도 조건을 추가했다.

4.3 벡터 거리 임계값(Distance Threshold)

이 벤치마크의 핵심 특징은 top-k 검색이 아닌 범위 검색(range search)을 사용하는 것이다:

```
ps_embedding <-> ${image_embedding} < ${D}
```

여기서 **<->** 는 유클리드 거리 연산자이고, **D** 는 사용자가 정한 임계값이다. top-k는 항상 정해진 k개를 반환하지만, 범위 검색은 임계값 D에 따라 반환 건수가 달라진다. 이 때문에 카디널리티 추정이 더 어렵고, 옵티마이저에 더 큰 도전 과제가 된다.

4.4 TPC-DS 확장

TPC-DS의 **item** 테이블에 **i_embedding** 컬럼을 추가하고, Q7, Q12, Q19, Q20, Q42, Q72, Q98을 선택하여 벡터 조건을 추가했다.

5. Exqutor 시스템 설계 (Section V)

5.1 전체 구조 (Figure 3)

Exqutor는 기존 범용 벡터 데이터베이스의 **Planner/Optimizer** 단계에 통합된다. 동작 흐름:

1. VAQ가 들어오면, Parser → Analyzer를 거쳐 **원래 실행 계획(Original Plan)**이 생성된다.
2. 이 계획이 Exqutor로 전달된다.
3. Exqutor는 벡터 인덱스가 있는지 없는지에 따라 두 가지 방법 중 하나로 **벡터 검색의 카디널리티를 계산한다**.
4. 계산된 카디널리티가 옵티마이저에 반환되어 **재최적화된 실행 계획(Re-optimized Plan)**이 생성된다.

5.2 방법 A: 벡터 인덱스가 있을 때 — ECQO (Exact Cardinality Query Optimization)

벡터 인덱스(예: HNSW)가 존재하는 경우, Exqutor는 쿼리 **계획 수립 단계**에서 인덱스를 이용한 가벼운 벡터 검색을 실제로 실행한다.

동작 원리:

1. 실제 쿼리 벡터와 사용자가 지정한 유사도 임계값을 사용해서, HNSW 인덱스에 범위 검색(range query)을 수행한다.
2. 검색 결과로 나온 **정확한 후보 벡터 수**를 카디널리티로 사용한다.
3. 이 정확한 카디널리티를 옵티마이저의 비용 모델에 반영하여, 조인 순서, 조인 방식, 스캔 전략 등을 결정한다.

왜 오버헤드가 작은가:

- HNSW 같은 인덱스는 계층적 그래프 탐색으로 동작하기 때문에, 계획 수립 단계에서 호출해도 매우 빠르게 완료된다 (보통 1~2ms 수준).
- 또한 계획 단계에서 얻은 검색 결과를 **캐시**해두고, 실제 실행 단계에서 **재사용**한다. 즉 인덱스 검색을 두 번 하지 않는다.
- HNSW는 같은 인덱스 구조, 같은 파라미터(`ef_search`), 같은 쿼리 벡터에 대해 **결정론적(deterministic)** 결과를 반환하므로, 캐시 재사용이 정확하다.

pgvector, VBASE에서의 구현:

플래너의 훅(hook)을 확장하여, 벡터 범위 조건(predicate)을 인식하고 인덱스 검색을 트리거한다. 검색 결과는 캐시되어 실행 단계에서 재사용된다.

DuckDB에서의 구현:

논리적 옵티마이저 규칙을 수정하여 카디널리티 추정을 통합했다. DuckDB의 인프로세스(in-process) 아키텍처 덕분에 이 추정값이 조인 재정렬, 스캔 전략 선택 등 하위 옵티마이저 단계로 효율적으로 전파된다.

5.3 방법 B: 벡터 인덱스가 없을 때 — 샘플링 기반 카디널리티 추정

벡터 인덱스가 없으면 전체 데이터를 스캔해야 하므로 ECQO를 사용할 수 없다. 이 경우 Exqutor는 **샘플링 기반 접근법**을 사용한다.

기본 아이디어:

데이터의 일부(샘플)만 추출해서, 그 샘플에서 유사도 조건을 만족하는 비율을 계산하고, 이를 전체 데이터에 대한 카디널리티 추정으로 활용한다.

초기 샘플 크기 결정 (수식 1):

$$N = \text{ceil}((z^2 \times P_{\text{hat}} \times (1 - P_{\text{hat}})) / e^2)$$

- z : 신뢰 수준에 해당하는 임계값 (95% 신뢰도 $\rightarrow z = 1.96$)
- P_{hat} : 유사도 임계값을 만족하는 데이터의 예상 비율 (기본값 0.5)
- e : 허용 오차 (기본값 0.05, 즉 5%)
- 이 수식으로 계산하면 $N = 385$ 가 나온다.

적응적 샘플 크기 조정(Adaptive Sampling):

고정 샘플 크기는 모든 데이터셋에 최적이지 아닐 수 있다. 예를 들어, 고차원이거나 편향된 분포를 가진 데이터에서는 샘플이 너무 크거나 작을 수 있다. 이를 해결하기 위해 Exqutor는 **모멘텀 기반 적응적 조정 알고리즘**을 사용한다.

핵심 지표로 **Q-error**를 사용한다:

$$Q\text{-error} = \max(\text{Card_esti} / \text{Card_true}, \text{Card_true} / \text{Card_esti})$$

Q-error는 추정값과 실제값의 비율 중 큰 쪽을 취한 것으로, 1에 가까울수록 정확한 추정이다.

조정 규칙 (수식 3~5):

1. Q-error와 현재 샘플링 비율로부터 조정 인자(δ)를 계산한다.
2. 모멘텀($m = 0.9$)을 적용해서 급격한 변동을 방지한다.
3. 학습률(η)을 감소시켜 점차 안정적으로 수렴한다.
4. 50개 쿼리마다 샘플 크기를 업데이트한다.

이 적응적 방식은 각 데이터셋의 특성에 맞게 자동으로 샘플 크기를 조절하며:

- DEEP, SimSearchNet++ 데이터셋: 시간이 지나면서 샘플 크기가 줄어든다 (안정적이므로)
- SIFT 데이터셋: 더 복잡한 분포를 가져서 샘플 크기가 증가한다

6. 실험 평가 (Section VI)

6.1 실험 환경

- **시스템:** Intel Xeon Gold 6530 (128 vCPU), 1.0 TB RAM
- **데이터베이스:** pgvector, VBASE, DuckDB(DuckDB-VSS)
- **벡터 데이터셋:** DEEP(96차원), SIFT(128차원), SimSearchNet++(256차원), YFCC(192차원), WIKI(768차원)
- **HNSW 파라미터:** M=16, ef_construction=200, ef_search=400
- **벤치마크:** TPC-H SF100 (Scale Factor 100), TPC-DS SF10

6.2 ECQO 성능 결과 (벡터 인덱스가 있는 경우)

Figure 4 결과 요약:

pgvector, VBASE, DuckDB 모두에서, Exqutor를 적용하면 쿼리 실행 시간이 크게 단축된다.

- **pgvector:** 최대 약 1,000배(3 orders of magnitude) 속도 향상
- **VBASE:** 최대 약 10,000배(4 orders of magnitude) 속도 향상
- **DuckDB:** 1.5배~37.2배 속도 향상

왜 이런 개선이 가능한가 (Figure 11 예시):

pgvector에서 Q3 쿼리를 DEEP 데이터셋으로 실행한 경우:

- **Exqutor 적용 전:** Full Table Scan + Hash Join 3번 → 총 52,535초 (약 14.6시간!)
- **Exqutor 적용 후:** HNSW Index Scan + Nested Loop Join → 총 15.571ms (0.016초)

개선 메커니즘:

1. 정확한 카디널리티로 인해 옵티마이저가 **Nested Loop Join**을 선택 (적은 결과에 적합)
2. **벡터 필터를 먼저 적용**해서 초기에 데이터를 대폭 줄임
3. Sequential Scan 대신 **Index Scan**을 선택
4. ECQO 자체의 오버헤드는 약 1.88ms에 불과

6.3 샘플링 기반 추정 성능 (벡터 인덱스가 없는 경우)

Figure 5 결과 요약:

- **고정 샘플링:** 기본 pgvector 대비 1.2배~3.2배 속도 향상
- **적응적 샘플링:** 고정 샘플링 대비 추가로 최대 1.4배 속도 향상

적응적 샘플링이 더 나은 이유:

고정 크기 샘플은 데이터 분포나 차원을 고려하지 않는다. 밀집 클러스터가 있거나 고차원인 경우, 고정 샘플이 선택도를 잘못 대표할 수 있다. 적응적 샘플링은 Q-error 피드백에 따라 샘플 크기를 자동 조절하여 일관된 성능을 보인다.

6.4 다양한 워크로드에서의 성능

태그 기반 필터링 + 벡터 검색 (Figure 7):

YFCC 데이터셋에서 태그 필터와 벡터 유사도 조건을 결합한 쿼리에서 최대 301.5배 속도 향상.

멀티 벡터 쿼리 (Figure 8, 9):

partsupp 테이블에 DEEP, part 테이블에 WIKI 벡터를 각각 저장하고 조인하는 복잡한 시나리오에서도 1.07배~479.4배 속도 향상.

TPC-DS 기반 평가 (Figure 10):

더 복잡한 TPC-DS 쿼리에서도 최대 109.6배 속도 향상. TPC-H와 유사한 쿼리 계획 변환 패턴이 관찰됨.

6.5 기존 방법과의 비교 (Figure 12)

SelNet(학습 기반 선택도 추정기)과 비교한 결과, Exqutor가 최대 16.1배 더 빠르다. SelNet은 오프라인 학습이 필요하고, 데이터 크기가 커지면 학습 비용이 증가하며, 외부 모델 관리 복잡성도 있다. 반면 Exqutor는 추가 학습 없이 즉시 사용 가능하다.

6.6 오버헤드 분석 (Table II)

ECQO의 오버헤드는 매우 작다:

- DEEP: 1.88ms 오버헤드로 43.34초 단축 (상대 오버헤드 0.0043%)
- SIFT: 1.89ms 오버헤드로 41.65초 단축 (상대 오버헤드 0.0045%)
- SimSearchNet++: 1.96ms 오버헤드로 43.66초 단축 (상대 오버헤드 0.0045%)

샘플링 기반 추정의 오버헤드는 28~73ms 수준으로, ECQO보다는 크지만 여전히 26~47초의 실행 시간 단축과 비교하면 미미하다.

6.7 확장성 (Figure 14)

Scale Factor를 1, 10, 100으로 변경하며 테스트:

- pgvector 기본: 데이터 크기에 비례해 거의 선형적으로 실행 시간 증가
- Exqutor 적용: HNSW 검색이 데이터 크기에 선형적으로 증가하지 않으므로, 안정적인 성능 유지. 최대 1.31배 속도 향상을 유지.

6.8 한계점

1. 고차원 공간에서 샘플링 오버헤드가 커질 수 있다 (거리 계산 비용 증가).
2. 기존 데이터베이스의 **비용 모델**이 ANN 인덱스의 성능 특성을 제대로 반영하지 못해, 정확한 카디널리티가 있어도 최적 계획을 선택하지 못할 수 있다.
3. 고차원에서의 효율적 샘플링과 벡터 인덱스에 맞는 정제된 비용 모델 개발이 향후 과제이다.

7. 관련 연구 (Related Work)

7.1 필터링된 벡터 검색

Milvus, Qdrant 등은 벡터 임베딩 옆에 메타데이터를 저장해서 필터링을 지원한다. ACORN, SeRF, HQANN, diskANN 등은 ANN 인덱스에 속성 필터링을 통합하지만, 모두 단일 테이블 내 검색에 한정되어 있어 복잡한 조인 쿼리에는 적용할 수 없다.

7.2 범용 벡터 DB의 쿼리 최적화

AnalyticDB는 비용 기반 모델을 사용하고, SingleStore는 인덱스 기반 필터 통합을 제공하지만, 이들은 단순한 필터 쿼리에 집중하며 복잡한 멀티 조인, 중첩 쿼리에는 효과적이지 않다.

7.3 샘플링 기반 선택도 추정

조인 크기 추정을 위한 랜덤 샘플링, 적응적 샘플링 전략 등 기존 연구가 있지만, 벡터 유사도 검색에 특화된 샘플링 기반 추정 은 Exqutor가 최초이다.

8. 결론 (Conclusion)

Exqutor는 벡터 증강 분석 쿼리(VAQ)의 성능을 향상시키기 위한 확장 쿼리 옵티마이저이다. 핵심 기여는:

1. **ECQO**: 벡터 인덱스를 활용한 정확한 카디널리티 추정으로, 최대 4 orders of magnitude(10,000배)의 속도 향상 달성.
2. **적응적 샘플링**: 벡터 인덱스가 없는 환경에서도 모멘텀 기반 동적 샘플 크기 조정으로 1.2~3.2배 속도 향상.
3. **VAQ 벤치마크**: TPC-H/TPC-DS를 확장한 최초의 벡터+구조화 데이터 통합 벤치마크 제공.
4. **범용성**: pgvector, VBASE, DuckDB 세 가지 서로 다른 아키텍처의 시스템에 성공적으로 통합.

오픈소스로 공개: <https://github.com/BDAI-Research/Exqutor>

9. 핵심 용어 정리

용어	설명
RAG	Retrieval-Augmented Generation. LLM이 답변 전에 관련 문서를 검색해서 참고하는 기술
VSS	Vector Similarity Search. 벡터 간 거리/유사도를 기반으로 유사한 데이터를 찾는 연산
VAQ	Vector-Augmented Analytical Query. 벡터 검색 + 관계형 연산(조인, 필터, 집계)이 결합된 쿼리
카디널리티	어떤 연산의 결과로 나오는 행(row)의 수. 쿼리 계획 수립에 핵심적인 정보
선택도 (Selectivity)	전체 데이터 중 조건을 만족하는 비율. 선택도 $\times$ 전체 행 수 = 카디널리티
HNSW	Hierarchical Navigable Small World. 계층적 그래프 기반 ANN 인덱스
ECQO	Exact Cardinality Query Optimization. 계획 단계에서 실제 쿼리 일부를 실행해 정확한 카디널리티를 얻는 방법
Q-error	추정 카디널리티와 실제 카디널리티의 비율로, 추정 정확도를 측정하는 지표
TPC-H / TPC-DS	데이터베이스 성능 평가를 위한 표준 벤치마크